List

本题的背景是编写一个简单的集中交易撮合函数（match）。

卖出订单 sells 和买入订单 buys 都是订单（order）的列表。订单本身也用一个两个元素的列表 [price, quantity] 表示

```python
sells = [[6, 2], [8, 3]]
buys = [[9, 3], [7, 2]]
```

List

本题的背景是编写一个简单的集中交易撮合函数（match）。

卖出订单 sells 和买入订单 buys 都是订单（order）的列表。订单本身也用一个两个元素的列表 [price, quantity] 表示

```python
sells = [[6, 2], [8, 3]]
buys = [[9, 3], [7, 2]]
```

例如，sells 中的 [6, 2] 表示卖家愿意以每瓶至少 6 枚金币的价格卖出 2 瓶药水；buys 中的 [9, 3] 表示买家愿意以每瓶至多 9 枚金币的价格买入 3 瓶药水。

订单的顺序

为了方便大家做题，已经提前将两组订单列表排好序：买入订单按价格从高到低排列，卖出订单按价格从低到高排列。因此，两个列表的**第一个元素**分别就是买入价格最高和卖出价格最低的订单。

```
1  sells = [[6, 2], [8, 3]]
2  buys = [[9, 3], [7, 2]]
```

卖单 sells

价格	数量
8	3
6	2

9	3
7	2
价格	数量

买单 buys

一次交易的步骤

随后，试卷中提供了进行一次交易的算法，分为三个步骤。我们关注两个列表的第一个元素，也就是价格最高的买单和价格最低的卖单：

1. 如果任意一个订单列表为空，或者最高买价低于最低卖价，就不能成交，不执行后续步骤。
2. 否则，就可以进行一笔交易，按卖价成交。成交数量为买单和卖单数量中的较小值，两单的数量都减去这个值。
3. 如果买单或卖单的数量变为 0，就将该订单从对应的列表中移除。

一次交易

原始订单		扣减成交数量		移除零数量订单	
卖单 sells		**卖单 sells**		**卖单 sells**	
价格	数量	价格	数量	价格	数量
8	3	8	3	8	3
6	2	6	**0**		
9	3	9	**1**	9	1
7	2	7	2	7	2
价格	数量	价格	数量	价格	数量
买单 buys		**买单 buys**		**买单 buys**	

成交 trade

`[6, 2]`

Table 2: 一次交易

实现 trade

trade 函数已经假设两个订单列表都非空，且最高买价不低于最低卖价。

```python
def trade(buys, sells):
    """Make one trade and change both order lists."""
    quantity = min(buys[0][1], sells[0][1]) # 成交数量为买单和卖单数量中的较小值
    price = sells[0][0]                     # 按卖价成交
    buys[0][1] -= quantity                  # 两单的数量都减去成交数量
    sells[0][1] -= quantity                 # 两单的数量都减去成交数量
    for orders in [buys, sells]:            # 买单或卖单
        if orders[0][1] == 0:               # 数量为 0
            orders.pop(0)
    return [price, quantity]
```

实现 match

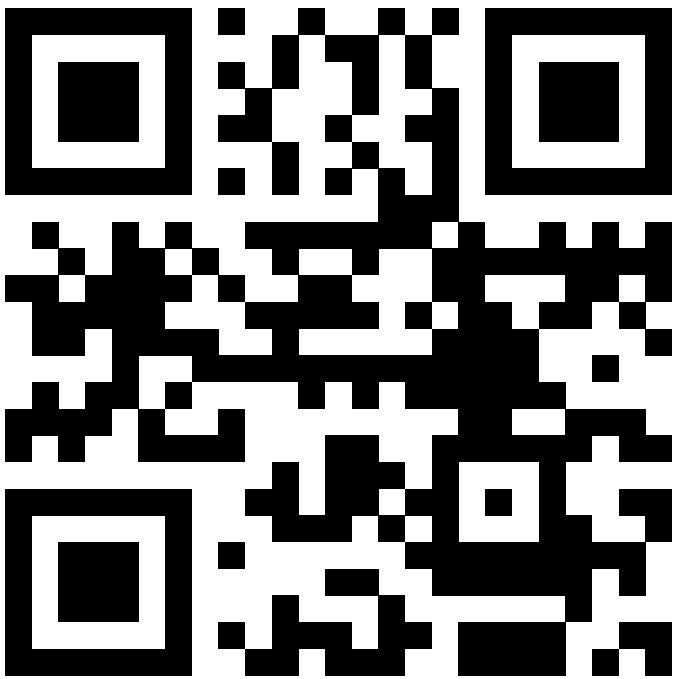
接下来一道小题要求实现一次集中交易撮合，也就是反复调用 `trade`，直到不再满足成交条件。

```python
def match(buys, sells):
    """Make all possible trades."""
    trades = []
    # 如果任意一个订单列表为空，或者最高买价低于最低卖价，就不能成交
    while buys and sells and buys[0][0] >= sells[0][0]:
        trades.append(trade(buys, sells)) # 进行一次交易
    return trades
```

列表的共享

下一道题考查的是列表的共享。

```python
>>> buys = [[9, 3], [7, 2]]
>>> sells = [[6, 2], [8, 3]]
>>> result = match(buys[:], sells[:])
>>> buys
[[9, 0], [7, 2]]
>>> sells
[[6, 0], [8, 2]]
```



Python Tutor

Copy Orders

`copy_orders` 这种深拷贝我们在 Lab04 的 Problem 3: Deep 中做过：复制外层列表，也复制每个订单列表。这里订单的元素都是整数，复制这两层就足够了。

```python
def copy_orders(orders):
    return [order[:] for order in orders]
```

The Ledger

最后是一道考查 list comprehension 的题，计算交易列表 `trades` 中的总数量和总价。

Python

```python
def summary(trades):
    """Return [total quantity, total cost].
    >>> summary([[6, 2], [8, 1]])
    [3, 20]
    """
    total = sum([q for p, q in trades])
    cost = sum([p * q for p, q in trades])
    return [total, cost]
```

Generators

Generator 这题通过三个步骤构建一个可以按宽度截断的分行器。

这里选取的是 POSIX 标准中的定义 3.185 Line：

> A sequence of zero or more non-<newline> characters plus a terminating <newline> character.



- <newline> 就是 '\n'。
- POSIX 标准中的 Line 包含 <newline>，但我们的输出不包含。
- 本题还会保留输入末尾未以 '\n' 结束的非空内容。

Line Splitter

Python

```python
def line_splitter(chars):
    """Yield each line as a string."""
    buffer = ''                     # 保存当前行已经读到的字符
    for char in chars:
        if char == '\n':            # 换行符结束当前行
            yield buffer            # yield 当前行，不包含换行符
            buffer = ''             # 清空 buffer，开始下一行
        else:
            buffer += char          # 普通字符加入当前行
    if buffer:                      # 如果还剩下一个不完整行
        yield buffer                # yield 不完整行
```

Width Splitter

`width_splitter` 按宽度分割，这里还额外考查了 slicing。

```python
def width_splitter(line, width):
    """Yield strings of at most width characters."""
    while len(line) > width:
        yield line[:width]      # yield 前 width 个字符
        line = line[width:]     # 剩余字符留到后续处理
    yield line                  # yield 最后一段，可能是空字符串
```

Line Width Splitter

组合两个 splitter，得到完整的 `line_width_splitter`。

```python
def line_width_splitter(chars, width):
    """Yield each line as a string, with at most width characters."""
    for l1 in line_splitter(chars):          # 先按换行符分行
        for l2 in width_splitter(l1, width): # 再按宽度分行
            yield l2
```

Line Width Splitter

组合两个 splitter，得到完整的 `line_width_splitter`。

```python
def line_width_splitter(chars, width):
    """Yield each line as a string, with at most width characters."""
    for l1 in line_splitter(chars):          # 先按换行符分行
        for l2 in width_splitter(l1, width): # 再按宽度分行
            yield l2
```

这里不能反过来，因为 `'\n'` 在 `width_splitter` 里也算宽度。

读取了多少字符?

最后判断 next(reader) 看了多少字符，才输出第一个按宽度截断的行。

Python

```
>>> seen = ''
>>> def source():
...     global seen
...     for char in "abcdef\ngh":
...         seen += char
...         yield char
>>> reader = line_width_splitter(source(), 2)
>>> next(reader)
'ab'
>>> seen
'abcdef\n'
```

读取了多少字符?

必须等 `line_splitter` 读到第一行末尾的 `'\n'`，yield 出 `l1` 后，才会执行下一步的 `width_splitter`。

```python
def line_width_splitter(chars, width):
    """Yield each line as a string, with at most width characters."""
    for l1 in line_splitter(chars):          # 先按换行符分行
        for l2 in width_splitter(l1, width): # 再按宽度分行
            yield l2
```

所以此时 `seen` 包含第一行及其末尾的 `'\n'`，即 `'abcdef\n'`，共读取了 7 个字符。